

Chapter 8. Container Orchestration with OpenShift and Kubernetes





Outline

- Deploy an application in OpenShift.
- Deploy a multi-pod application to OpenShift and make it externally available.

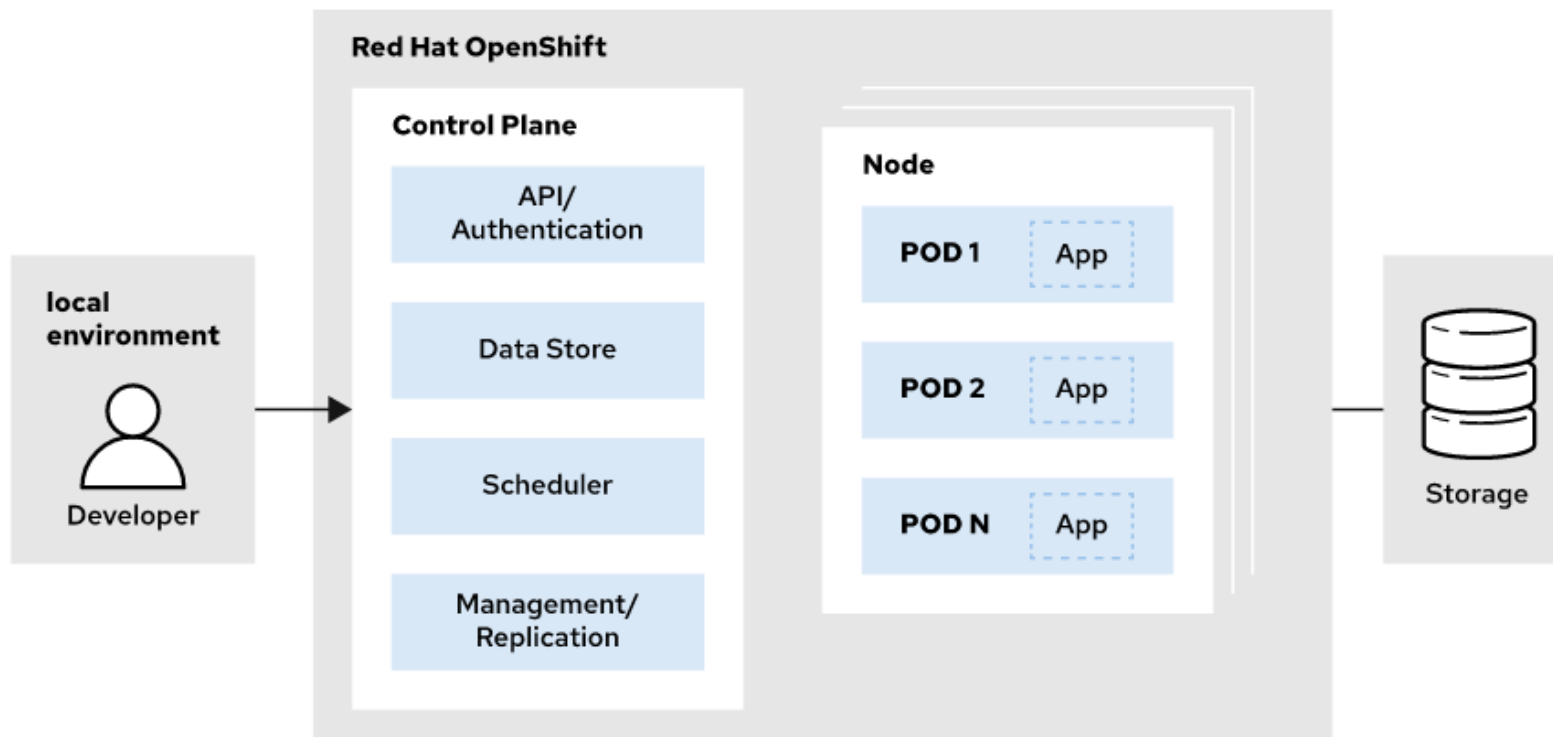


Kubernetes and RHOCP architecture

- Kubernetes is an **orchestration platform** that **simplifies the deployment, management, and scaling of containerized applications**.
- Kubernetes uses several **servers** (also called **nodes**) to ensure the resiliency and scalability of the applications it manages.
- Kubernetes forms a **cluster of servers** that run containers and are centrally **managed by a set of control plane servers**. A server can act as both a control plane node and a compute node, but those roles are usually separated for increased stability, security, and manageability.
- **Red Hat OpenShift Container Platform (RHOCP)** is a set of modular components and **services that use and expand Kubernetes**.
- **RHOCP** adds capabilities such as **remote management, increased security, monitoring and auditing, application lifecycle management, and self-service interfaces** for developers.



Kubernetes and RHOCP architecture





Kubernetes and RHOC P architecture

- Developers **deploy application** containers **grouped in pods**.
- **Pods** are **RHOC P objects** that represent the **smallest deployment unit managed by RHOC P**.
- Developers **group related pods** and **other RHOC P objects** in **projects**.
- **A project** is a **RHOC P resource** which is similar to the **namespace Kubernetes resource**.



Interacting with RHOCP with the Web Console

- **RHOCP web console** is a **browser-based user interface** you can use to interact with RHOCP. You can deploy and manage your applications by using the web console.
- **Web console provides the Administrator and Developer perspectives.**
 - 1) **Developer perspective:** focuses on the status and management of the **deployed applications**.
 - 2) **Administrator perspective:** focuses on the status and management **of RHOCP and its resources**.



Interacting with RHOCP with CLI

- **Command-line utilities (CLI)** provide an **alternative for interacting with your RHOCP cluster**.
- Developers familiar with Kubernetes can **use the kubectl utility** to manage a RHOCP cluster. This course uses the **oc command-line utility**, which is designed to take advantage of additional RHOCP features.
- The CLI utilities provide developers with a range of commands that are useful for managing the RHOCP cluster and its applications. Each command is translated into an API call, and the response is displayed in the command line.



Deploy Applications as Pods to RHOCP

- In RHOCP, pods are the atomic unit for deploying and managing your applications. For example, you can deploy a single-container application by creating a pod that contains a single container.
- However, pods do not expose advanced functionality such as application scaling or zero-downtime updates. Such functionality is implemented in a controller object that manages a set of pods, for example the **deployment object**.
- A **Pod** is the **smallest deployable unit** that can be deployed and managed by Kubernetes.
- In other words, if you need to **run a single container in Kubernetes**, then you need to create a Pod for that container.
- At the same time, a **Pod can contain more than one container**, if these containers are **relatively tightly coupled**.
- Pods are not visible outside k8s cluster.



Declarative way

- **Storing a pod definition** in a file is called the **declarative approach** for creating RHOCP objects, because you declare the intended pod state outside of RHOCP.
- This means you can store the pod definition in a version control system, such as Git, and incrementally change your definition as your application evolves.

```
kind: Pod ①
apiVersion: v1
metadata:
  name: example-pod ②
  namespace: example-project ③
spec: ④
  containers: ⑤
  - name: example-container ⑥
    image: quay.io/example/awesome-container ⑦
    ports: ⑧
    - containerPort: 8080
    env: ⑨
    - name: GREETING
      value: "Hello from the awesome container"
```



Imperative way

- For testing purposes, RHOCP also provides the **imperative approach** to creating RHOCP objects. The imperative approach uses the **oc run command** to create a pod without a definition.
- The imperative commands are a **faster way of creating pods**, because such commands **do not require a pod object definition**. However, developers lose the ability to version and incrementally change the pod definition.
- Generally, developers test the deployment by using imperative commands, and then use the imperative commands to generate the pod object definition. Use the **--dry-run=client** option to avoid creating the object in RHOCP. Additionally, use the **-o yaml** or **-o json** option to configure the definition format.
- **oc run example-pod --image=quay.io/example/awesome-container --env GREETING='Hello from the awesome container' --port=8080 --dry-run=client -o yaml**



Application Networking in RHOCP

- Developers configure internal pod-to-pod network communication in RHOCP by using the **Service object**.
- Applications send requests to the **service name** and **port**. RHOCP provides a virtual network which reroutes such requests to the pods that the service targets by using labels.

Declaratively

```
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  ports:
    - port: 8080 1
      protocol: TCP
      targetPort: 8080 2
  selector: 3
    app: backend-app
```

Imperatively

```
oc expose pod backend-app --port=8080
--target-port=8080 --name=backend-app
--dry-run=client -o yaml
```

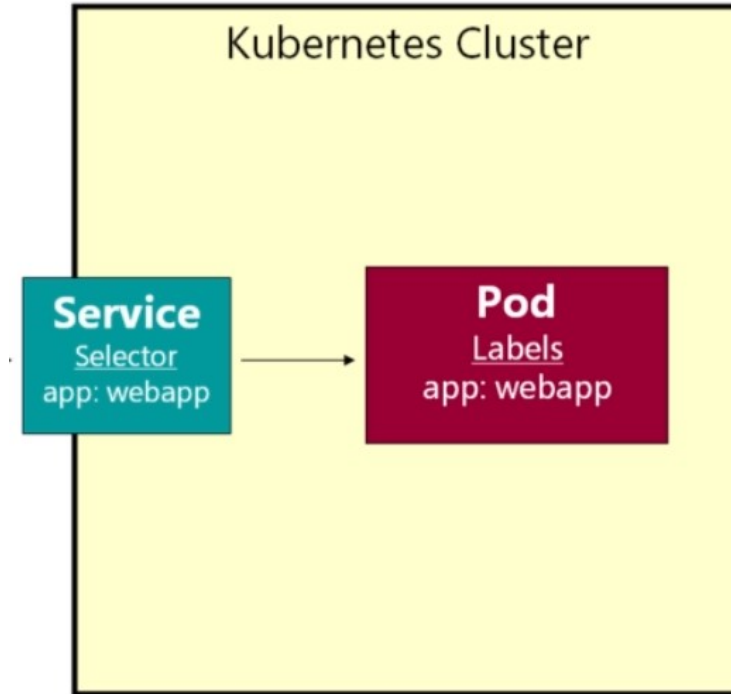
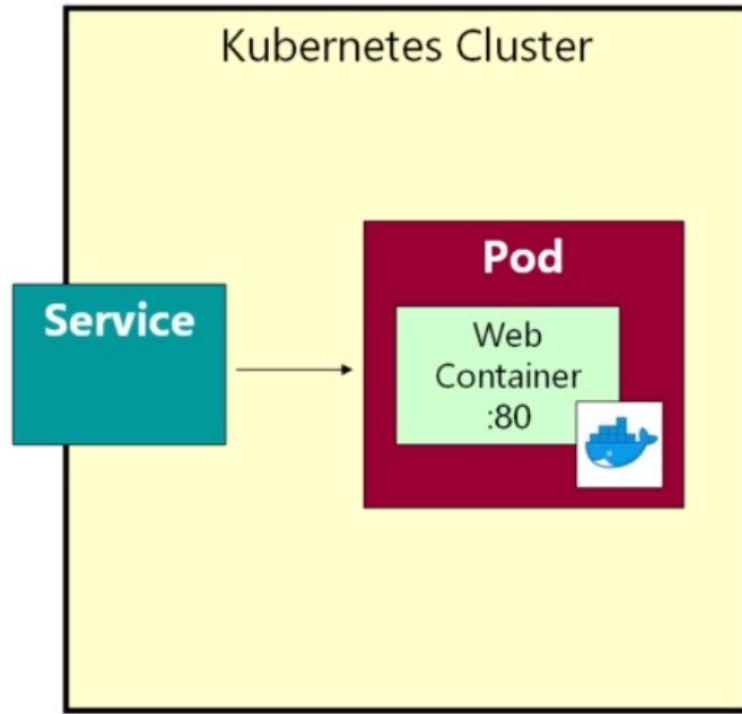


Types of Service

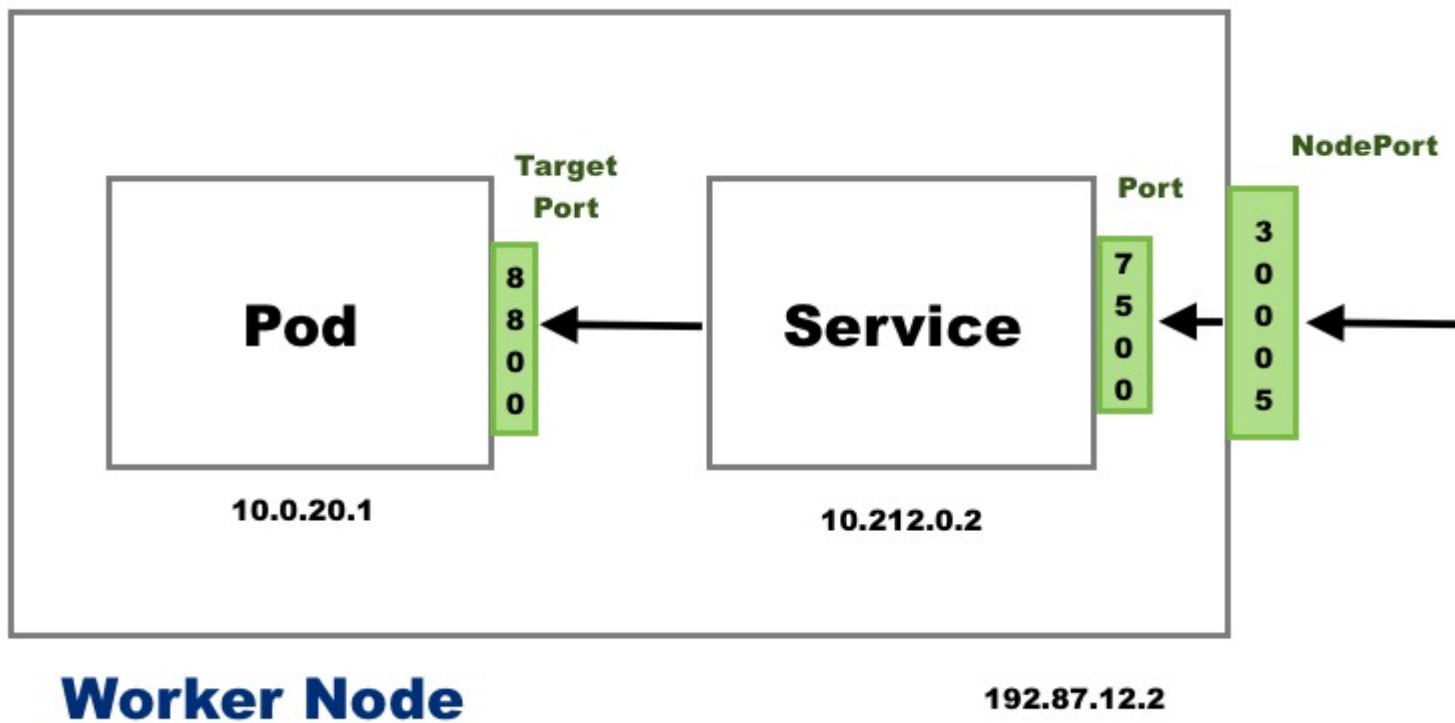
- 1) ClusterIP:** the ClusterIP Service is **used for Pod-to-Pod communication within the same cluster**. This means that a client running outside of the cluster, such as a user accessing an application over the internet, cannot directly access a ClusterIP Service.
- 2) NodePort:** The NodePort Service is a way to **expose your application to external clients**. An external client is anyone who is trying to access your application from outside of the Kubernetes cluster. The NodePort Service does this by opening the port you choose (in the range of 30000 to 32767) on all worker nodes in the cluster. This port is what external clients will use to connect to your app.
- 3) LoadBalancer:** A LoadBalancer Service is another way you **can expose your applications to external clients**. However, it only works if you're using Kubernetes on a **cloud platform that supports this Service type**.



Application Networking in RHOCP



Application Networking in RHOCP





ReplicaSet

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

Template





List of the most common oc commands

<code>oc login https://api.ocp4.example.com:6443</code>	Use the login command to authenticate your requests.
<code>oc create -f pod.yaml</code>	Create resources from the pod.yaml file,
<code>oc get <resource-type></code> EX: <code>oc get pod</code>	Returns the list of the pod resources in the current project.
<code>oc delete <resource-type> <resource-name></code> EX: <code>oc delete pod <pod-name></code> <code>oc delete deployment <deployment-name></code>	Delete an existing RHOC P resource. You must specify the resource type and the resource name.
<code>oc logs <pod-name></code>	You can print only logs of a container in a pod, which means the resource type is omitted.



List of the most common oc commands

<code>oc explain pod.metadata.name</code>	Get information about valid fields for an object.
<code>oc run example-pod \</code> <code>--image=quay.io/example/awesome-</code> <code>container \</code> <code>--env GREETING='Hello from the awesome</code> <code>container' \</code> <code>--port=8080 \</code> <code>--dry-run=client -o yaml</code>	Use the imperative commands to generate the pod object definition. Use the <code>--dry-run=client</code> option to avoid creating the object in RHOCP. Additionally, use the <code>-o yaml</code> or <code>-o json</code> option to configure the definition format.
<code>oc expose pod backend-app \</code> <code>--port=8080 \</code> <code>--targetPort=8080 \</code> <code>--name=backend-app</code>	Create services imperatively for a specific pod



List of the most common oc commands

```
oc create deployment example-deployment \  
  --image=quay.io/example/awesome-  
container \  
  --replicas=3 \  
  --dry-run=client -o yaml
```

use the oc create deployment command to create a deployment. with --dry-run=client and -o options to generate a deployment definition.

```
oc expose pod backend-app \  
  --port=8080 \  
  --targetPort=8080 \  
  --name=backend-app
```

Create services imperatively for a specific pod

```
oc command --help
```

Section 7.3. Multi-pod Applications





Manage Pods with Controllers

- Kubernetes encourages developers to use a **declarative deployment style**, which means that the developer declares the desired application state and Kubernetes reaches the declared state. Kubernetes uses the **controller pattern** to implement the **declarative deployment style**.
- In the controller pattern, a **controller** continuously **watches the current state of a system**. If the **system state does not match the desired state**, then the controller sends signals to the system until the **system state matches the desired state**. This is called the **control loop**.
- Kubernetes uses a number of controllers to deal with constant change.
- Consider a single-container bare pod application, which means an application that is deployed by using the Pod object. **When a bare pod application fails**, for example due to a memory leak, Kubernetes handles the failure event by restarting the pod.
- However, in case of a node failure, Kubernetes does not reschedule a bare pod, which is why developers rarely use bare pods for application deployment.
- You can use a number of controller objects provided by Kubernetes, such as **Deployment**, **ReplicaSet**, **StatefulSet**, and others.



Deploy Pods with Deployments

- The **Deployment object** is a **Kubernetes controller that manages pods**. Developers use deployments for the following use cases:

1) Managing Pod Scaling

You can configure the number of pod replicas for a given deployment. If the actual replica count decreases, for example due to node failure or a network partition, then the deployment schedules new pods until the declared replica count is reached.

2) Managing Application Changes

Most of the **Pod object** fields are **immutable**, such as the name or environment variable configuration. To change those fields with bare pods, developers must **delete the pod and recreate** it with the new configuration.

When you manage pods with a deployment, you declare the new configuration in the deployment object. The **deployment controller** then **deletes the original pods and recreates the new pods** that contain the new configuration.

3) Managing Application Updates

Deployments implement the **RollingUpdate strategy** for gradually updating application pods when you declare a new application image. **This ensures zero-downtime application updates.**

Create Deployments Declaratively

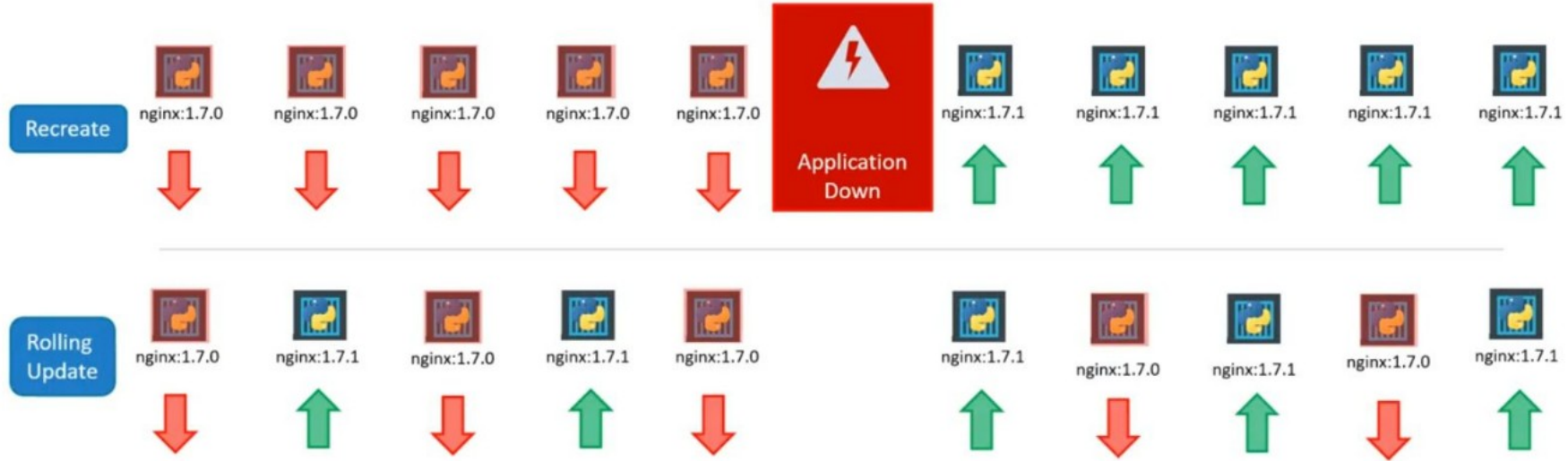


```
apiVersion: apps/v1
kind: Deployment
metadata: ❶
  labels:
    app: deployment-label
  name: example-deployment
spec:
  replicas: 3 ❷
  selector: ❸
    matchLabels:
      app: example-deployment
  strategy: RollingUpdate ❹
  template: ❺
    metadata:
      labels:
        app: example-deployment
    spec: ❻
      containers:
        - image: quay.io/example/awesome-container
          name: awesome-pod
```

- **Strategy**: specifies the strategy used to replace old Pods by new ones.
- **spec.strategy.type** can be "**Recreate**" or "**RollingUpdate**".
- **RollingUpdate**: is the default value.
- **Recreate Deployment**: All existing Pods are killed before new ones are created.
- The default **RollingUpdate** strategy ensures gradual, zero-downtime pod rollout when you modify the container image.



Deployment Strategy



Create Deployments Imperatively



- **oc create deployment <deployment-name> --image=<img-name> --replicas=3 --dry-run=client -o yaml**
- Controllers create and manage the lifecycle of pods that are specified in the controller configuration.
- Consequently, a controller must have an ownership relationship between pods, and one pod can be owned by at most one controller.
- The **deployment controller uses labels to target dependent pods.**

Expose Applications for External Access



- Developers commonly expose applications for access from outside of the Kubernetes cluster.
- The default Service object provides a stable, **internal IP address** and a **DNS record for pods**. However, applications outside of the Kubernetes cluster **cannot connect to the IP address or resolve the DNS record**.
- You can **use the Route object to expose applications**.
- **Route** is a Red Hat OpenShift Container Platform (**RHOCP**) **object** that connects a **public-facing IP address** and **DNS host name** to an **internal-facing IP address**. Routes use information from the service object to route requests to pods.
- RHOCP exposes an **HAProxy router pod** that listens on a RHOCP node public IP address. The router serves as an ingress entrypoint to the internal RHOCP traffic.

Create Routes Declaratively

Imperatively



```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  labels:
    app: app-ui
  name: app-ui
  namespace: awesome-app
spec:
  port:
    targetPort: 8080 ①
  host: ""
  to: ②
    kind: "Service"
    name: "app-ui"
```

- **oc expose service <svc-name> --dry-run=client -o yaml**

Create Routes Imperatively



- Note that you can use the `oc expose imperative` command in the following forms:
 - **`oc expose pod POD_NAME`**: create a service for a specific pod.
 - **`oc expose deployment DEPLOYMENT_NAME`**: create a service for all pods managed by a controller, in this case a deployment controller.
 - **`oc expose service SERVICE_NAME`**: create a route that targets the specified service.

Section 8.5: Lab: Container Orchestration with Kubernetes and OpenShift



Chapter 8 Task



- Write a YAML file named “nginx-pod.yaml” to define a pod named “nginx” from “nginx” image.
- Then create a service of type node port with port 80 to expose this “nginx” pod.

Chapter 8 Task



```
12003ad070221d ffff: secretKey was empty, can't make reference
Name:          node-pod
Namespace:     default
Priority:      0
PriorityClassName: <none>
Node:          minikube/192.168.49.2
Start Time:    Wed, 25 Dec 2024 23:18:23 +0200
Labels:        app=nyapp
Annotations:   kubectl.kubernetes.io/last-applied-configuration={"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"labels":{"app":"nyapp"},"name":"node-pod","namespace":"default"},"spec":{"containers":[{"name":"node-container","image":"marwaahmed11/node-app:v1.0","ports":[{"containerPort":30829}]}]}
Status:        Running
IP:            10.244.0.40
Containers:
  node-container:
    Container ID:  docker://4c1780e0a78032d6440efbb59d382daecf5306c21aca7d9f2e6e5aa761822ffd
    Image:         marwaahmed11/node-app:v1.0
    Image ID:      docker-pullable://marwaahmed11/node-app@sha256:19089e5260db6a93b4fee5e17a5b96b2507cbaef21151774611e83b5cf31729
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Wed, 25 Dec 2024 23:18:23 +0200
    Ready:         True
    Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-7rrjj (ro)
Conditions:
  Type            Status
  Initialized      True
  Ready            True
  ContainersReady  True
  PodScheduled     True
Volumes:
  kube-api-access-7rrjj:
    <unknown>
QoS Class:        BestEffort
Node-Selectors:   <none>
Tolerations:      node.kubernetes.io/not-ready:NoExecute for 300s
                  node.kubernetes.io/unreachable:NoExecute for 300s
marwa@marwa:~$ minikube ip
192.168.49.2
```

- k describe pod node-pod => to get node ip
- Node: node01/172.30.2.2
- http://<Node-IP>:<Node-Port> => node ip same as minikube ip
- curl http://172.30.2.2:30829